ΟΙ ΥΠΟΛΟΙΠΟΙ ΤΥΠΟΙ ΔΕΔΟΜΕΝΩΝ ΣΤΗΝ PYTHON

Μ.Ι.Σ. 2025-2026

Εκτός από τους 3 βασικούς τύπους δεδομένων,

- **αριθμοί,**
- **λογικοί** (boolean),
- **συμβολοσειρές** (strings),

η γλώσσα PYTHON διαθέτει επιπλέον και κάποιους σύνθετους τύπους δεδομένων,

- **λίστες** (lists)
- **πλειάδες** (tuples)
- **λεξικά** (dictionaries)

οι τύποι αυτοί είναι στην πραγματικότητα, <u>οι πίνακες</u>, που υπάρχουν στις C like γλώσσες.

Μ.Ι.Σ. 2025-2026

Ο ΤΥΠΟΣ ΔΕΔΟΜΕΝΩΝ - ΛΙΣΤΕΣ (lists)

Στην γλώσσα PYTHON η δομή **λίστας** (**list**) είναι μια διατεταγμένη συλλογή από τιμές, σε μορφή ενός μονοδιάστατου πίνακα όπου οι τιμές προσεγγίζονται με ένα δείκτη. Οι τιμές του πίνακα (της λίστας) ονομάζονται **στοιχεία** (**elements**).

Τα νεότερο με την PYTHON είναι ότι τα στοιχεία μιας λίστας δεν χρειάζεται να είναι ίδιου τύπου και επίσης ένα στοιχείο σε μία λίστα μπορεί να υπάρχει περισσότερες από μία φορές.

Επιπλέον μία λίστα μπορεί να περιέχεται μέσα σε μία άλλη λίστα και θα ονομάζεται **εμφωλιασμένη λίστα** (**nested list**). Αν μία λίστα περιέχει μέσα της μια άλλη λίστα (σαν στοιχείο), τότε η εμφωλιασμένη λίστα (ολόκληρο το στοιχείο) είναι σαν ένα απλό στοιχείο.

Οι λίστες ονομάζονται και **ακολουθίες** (**sequences**).

Όταν δηλώνουμε μια λίστα, τα στοιχεία της λίστας διαχωρίζονται με κόμμα και περικλείονται όλα ανάμεσα σε τετράγωνες αγκύλες **[** και **]**.

Μία άδεια λίστα που δεν περιέχει στοιχεία ονομάζεται **άδεια λίστα** και συμβολίζεται με **[]**.

Στην λίστα μπορούμε να μπούμε με δυο τρόπους, από την αρχή και από το τέλος
- εάν μπούμε από την αρχή, μετράμε στον δείκτη και το 0
- εάν μπούμε από το τέλος, δεν μετράμε στον δείκτη το 0

Για να προσπελάσουμε μια λίστα, χρησιμοποιούμε παρόμοια σύνταξη με αυτή της προσπέλασης των χαρακτήρων μιας συμβολοσειράς. Αν προσπελάσουμε στοιχείο που δεν υπάρχει, θα βγούμε με μήνυμα λάθους εκτός του πίνακα.

Μ.Ι.Σ. 2025-2026

Μπορούμε να διαπεράσουμε λίστες με την εντολή **for**, ενώ μπορούμε να βρούμε ένα στοιχείο με την εντολή **in** και **not in** ενώ με την συνάρτηση **len** βρίσκουμε το πλήθος των στοιχείων της λίστας. Μπορούμε να διαγράψουμε στοιχεία από μια λίστα με τον ειδικό τελεστή **del**.

Παράδειγμα 1,

εάν θέλουμε να δημιουργήσουμε μια απλή λίστα με ένα αύξοντα αριθμό, μπορούμε να γράψουμε,

```
my_pinakas = int(range(1,20))
```

η PYTHON γεμίζει έναν πίνακα (λίστα) με αριθμούς από το 1 έως το 20

Παράδειγμα 2,

με τον τελεστή + μπορούμε να ενώσουμε λίστες και

Παράδειγμα 3,

με τον τελεστή * μπορούμε να επαναλάβουμε μια λίστα, n φορές.

Μ.Ι.Σ. 2025-2026

ΛΙΣΤΕΣ ΣΕ 2 ΔΙΑΣΤΑΣΕΙΣ

Μπορούμε να αναπαραστήσουμε πίνακες 2 διαστάσεων με μια λίστα μέσα σε μια άλλη,

ο δυο διαστάσεων πίνακας

1	2	3
5	6	7
a	b	c

μπορεί να γραφεί σαν,

```
my_2d = [   [1, 2, 3], [5, 6, 7], [a, b, c]  ]
```

και αντίστοιχα θα έχουμε,

```
print(my_2d[1])
        [5, 6, 7]
print(my_2d[0])
        [1, 2, 3]
print(my_2d[0][2])
        [3]
print(my_2d[2][0])
        [a]
```

Το ίδιο μπορούμε να κάνουμε και για πίνακες 3 διαστάσεων με μια λίστα μέσα σε μια άλλη και μέσα σε μια άλλη.

Μ.Ι.Σ. 2025-2026

ΒΑΣΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ΓΙΑ ΛΙΣΤΕΣ (ΠΙΝΑΚΕΣ)

Η Python διαθέτει «εργοστασιακές» συναρτήσεις για επεξεργασία των περιεχομένων σε λίστες, διαφόρων χρήσεων,

- **my_ls.append(eee)** βάζει το eee στο τέλος της λίστας my_ls
- **my_ls.sort()** ταξινομεί τα περιεχόμενα της λίστας my_ls
- **my_ls.extend(new_ls)** επεκτείνει την λίστα my_ls με το περιεχόμενο της new_ls
- **my_ls.remove(eee)** διαγράφει από τα περιεχόμενα της λίστας my_ls το πρώτο eee
- **my_ls.pop(my_index)** αφαιρεί και επιστρέφει μέσα από την λίστα my_ls το περιεχόμενο στη θέση my_index

καθώς υπάρχουν πολλές σχετικές συναρτήσεις για τις λίστες και τον χειρισμό τους η εντολή **help(list)** θα τυπώσει όλες τις διαθέσιμες επιλογές.

πριν από κάθε μέθοδο/ρουτίνα/συνάρτηση, υπάρχει μια τελεία, η οποία σημαίνει, εκτέλεσε την ρουτίνα επάνω σε αυτό που προηγείται της τελείας (σε αυτή την λίστα δηλαδή).

Μ.Ι.Σ. 2025-2026

Ο ΤΥΠΟΣ ΔΕΔΟΜΕΝΩΝ - ΠΛΕΙΑΔΕΣ (tuples)

Στην γλώσσα PYTHON η δομή **πλειάδας** (**tuple**) είναι μια επίσης διατεταγμένη συλλογή από τιμές, σε μορφή ενός μονοδιάστατου πίνακα όπου οι τιμές προσεγγίζονται με ένα δείκτη. Οι τιμές του πίνακα (της πλειάδας) ονομάζονται επίσης **στοιχεία** (**elements**).

Τα στοιχεία μιας πλειάδας δεν χρειάζεται να είναι ίδιου τύπου και επίσης ένα στοιχείο σε μία πλειάδα μπορεί να υπάρχει περισσότερες από μία φορές.

Η διαφορά της **πλειάδας** σε σχέση με την **λίστα**, είναι ότι τα στοιχεία (το περιεχόμενο) της πλειάδας παραμένουν αμετάβλητα. Χρησιμοποιούνται για περιεχόμενο που δεν πρόκειται να αλλάξει. Στην επεξεργασία τους, ακολουθούμε τις ίδιες ενέργειες όπως και στις απλές λίστες.

Επιπλέον μία πλειάδα μπορεί να περιέχεται μέσα σε μία άλλη πλειάδα και θα ονομάζεται **εμφωλιασμένη πλειάδα** (**nested tuple**). Αν μία πλειάδα περιέχει μέσα της μια άλλη πλειάδα (σαν στοιχείο), τότε η εμφωλιασμένη πλειάδα (ολόκληρο το στοιχείο) είναι ένα απλό στοιχείο.

Οι πλειάδες ονομάζονται και **αμετάβλητες ακολουθίες**.

Όταν δηλώνουμε μια πλειάδα, τα στοιχεία μιας πλειάδας διαχωρίζονται με κόμμα και περικλείονται όλα ανάμεσα σε παρένθεση **(** και **)** ή απλά το κενό.

Μία άδεια **πλειάδα** που δεν περιέχει στοιχεία ονομάζεται **άδεια πλειάδα** και συμβολίζεται με **()**.

ΠΡΟΣΟΧΗ, Για να φτιάξουμε μια πλειάδα με ένα μόνο στοιχείο για περιεχόμενο, πρέπει να προσθέσουμε στο τέλος του στοιχείου ένα κόμμα. Χωρίς το κόμμα η γλώσσα θα θεωρήσει ότι δηλώνουμε απλή συμβολοσειρά μέσα σε παρενθέσεις.

Μ.Ι.Σ. 2025-2026

Παράδειγμα,

ΣΩΣΤΟ

```
my_tuple = ('DELTA',)
```

ΛΑΘΟΣ

```
my_tuple = ('DELTA')
```

Στην πλειάδα μπορούμε να «μπούμε» με δυο τρόπους, από την αρχή και από το τέλος

- εάν μπούμε από την αρχή, μετράμε στον δείκτη και το 0
- εάν μπούμε από το τέλος, δεν μετράμε στον δείκτη το 0

Για να προσπελάσουμε μια πλειάδα, χρησιμοποιούμε παρόμοια σύνταξη με αυτή της προσπέλασης των χαρακτήρων μιας συμβολοσειράς ή μιας λίστας. Αν προσπελάσουμε στοιχείο που δεν υπάρχει, θα βγούμε με μήνυμα λάθους εκτός του πίνακα.

Αν δοκιμάσουμε να αλλάξουμε κάποιο από τα περιεχόμενα μιας πλειάδας, θα βγούμε με μήνυμα λάθους. Μπορούμε όμως να αντικαταστήσουμε ολόκληρη την πλειάδα με κάποια διαφορετική. Αυτή η αντικατάσταση γίνεται με την ειδική εντολή ανάθεσης,

```
my_tuple_a, my_tuple_b = my_tuple_b, my_tuple_a
```

με αποτέλεσμα η α να πάρει τα περιεχόμενα της β και η β να πάρει τα περιεχόμενα της α.

Μ.Σ. 2025-2026

Στην γλώσσα PYTHON μπορούμε να περάσουμε σε μια συνάρτηση (σαν όρισμα) την δομή μιας πλειάδας. Το ίδιο μπορεί να γίνει και στην επιστροφή,

```
def  my_roytina(my_a, my_b):
        .
        .
        .
        return my_b, my_a
```

Οι πλειάδες στην γλώσσα PYTHON μπορούν να εφαρμοστούν και για $2^η$ και για περισσότερες διαστάσεις, όπως και στις απλές λίστες.

Μ.Ι.Σ. 2025-2026

Ο ΤΥΠΟΣ ΔΕΔΟΜΕΝΩΝ - ΛΕΞΙΚΑ (dictionary)

Στην γλώσσα PYTHON η δομή **λεξικό (dictionary**) είναι μια επίσης διατεταγμένη συλλογή από τιμές, σε μορφή ενός μονοδιάστατου πίνακα όπου οι τιμές συσχετίζονται με έναν **εσωτερικό ειδικό δείκτη**, που παίζει το ρόλο ενός μοναδικού πρωτεύοντος κλειδιού. Οι τιμές του πίνακα (του λεξικού) ονομάζονται **στοιχεία** (**elements**).

Τα στοιχεία του λεξικού δεν χρειάζεται να είναι ίδιου τύπου και επίσης ένα στοιχείο σε ένα λεξικό μπορεί να υπάρχει περισσότερες από μία φορές.

Η διαφορά του λεξικού σε σχέση με την **πλειάδα** και την **λίστα**, είναι ότι τα στοιχεία (το περιεχόμενο) του λεξικού συνδέονται χειροκίνητα με μια τιμή που παίζει το ρόλο του δείκτη-κλειδιού.

κλειδί : περιεχόμενο

η μοναδική τιμή κάθε κλειδιού αντιστοιχίζεται σε ένα μόνο περιεχόμενο

- για κλειδί μπορούμε να χρησιμοποιούμε <u>μόνο αμετάβλητα στοιχεία</u> (ακέραιο, συμβολοσειρά, πλειάδα)
- για περιεχόμενο μπορούμε να έχουμε <u>είτε αμετάβλητα ή μεταβλητά στοιχεία</u>

στα λεξικά δεν υπάρχει η έννοια της σταθερής θέσης του δείκτη, όπως στους πίνακες και έτσι σε ένα λεξικό δεν είναι δυνατό να γίνει πραγματική σειριακή προσέγγιση, καθώς δεν είναι διατεταγμένο, και ούτε μπορούμε να πάρουμε εγγραφές από ένα σημείο μέχρι κάποιο άλλο επόμενο σημείο.

Το περιεχόμενο τους μπορεί να αλλάζει, είναι δηλαδή μετατρέψιμα, και μπορούμε να προσθέσουμε νέα στοιχεία ή να διαγράψουμε κάποια στοιχεία που ήδη υπάρχουν.

Μ.Ε. 2025-2026

Στην επεξεργασία τους, ακολουθούμε περίπου τις ίδιες ενέργειες όπως και στις απλές λίστες και τις πλειάδες.

Για παράδειγμα, για να δημιουργήσουμε άδειο ένα λεξικό θα δώσουμε,

```
my_first_lexiko = {}
```

για να το γεμίσουμε

```
my_first_lexiko['0'] = 'ΑΘΗΝΑ'
my_first_lexiko['1'] = 'ΠΕΡΙΣΤΕΡΙ'
my_first_lexiko['2'] = 'ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ'
```

ή

```
my_first_lexiko['ΑΡΧΗ']        = 'ΑΘΗΝΑ'
my_first_lexiko['ΕΡΟΜΕΝΟ'] = 'ΠΕΡΙΣΤΕΡΙ'
my_first_lexiko['2']              = 'ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ'
```

ή

```
my_first_lexiko =  { 'ΑΡΧΗ' : 'ΑΘΗΝΑ',  'ΕΡΟΜΕΝΟ' : 'ΠΕΡΙΣΤΕΡΙ', '2' : 'ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ' }
```

Μ.Ι.Σ. 2025-2026

εάν τυπώσουμε τον πρώτο πίνακα-**λεξικό** θα έχουμε,

{ ‘0’ : ‘ΑΘΗΝΑ’, ‘1’ : ‘ΠΕΡΙΣΤΕΡΙ’, ‘2’ : ‘ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ’ }

εάν τυπώσουμε τον δεύτερο ή τον τρίτο πίνακα-**λεξικό** θα έχουμε,

{ ‘ΑΡΧΗ’ : ‘ΑΘΗΝΑ’, ‘ΕΡΟΜΕΝΟ’ : ‘ΠΕΡΙΣΤΕΡΙ’, ‘2’ : ‘ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ’ }

εάν τώρα γνωρίζουμε το κλειδί κάποιου από τα στοιχεία του πίνακα-**λεξικό,** μπορούμε να το τυπώσουμε με την εντολή

```
print(my_first_lexiko[‘ΕΡΟΜΕΝΟ’])
```

και θα δούμε το περιεχόμενο με βάση το δείκτη ‘ΕΡΟΜΕΝΟ’ , που είναι το

ΠΕΡΙΣΤΕΡΙ

εάν θέλουμε να σβήσουμε μέσα από το πίνακα-**λεξικό** μία εγγραφή μπορούμε να πληκτρολογήσουμε το,

```
del my_first_lexiko[‘ΕΡΟΜΕΝΟ’]
```

Μ.Ι.Σ. 2025-2026

με τον τελεστή **in** μπορούμε να ελέγξουμε εάν υπάρχει μέσα στο πίνακα-**λεξικό** μία εγγραφή,

'ΕΡΟΜΕΝΟ' in my_first_lexiko

και θα δούμε το,

False

Μ.Ι.Σ. 2025-2026

Γενικότερα, μπορούμε να δούμε μέσα σε ένα πίνακα-**λεξικό**

- τα κλειδιά, με την μέθοδο **keys**
- τα περιεχόμενα, με την μέθοδο **values**
- τα κλειδιά και το περιεχόμενο, με την μέθοδο **items**

ειδικότερα για τον,

```
my_first_lexiko =  { 'ΑΡΧΗ' : 'ΑΘΗΝΑ',  'ΕΡΟΜΕΝΟ' : 'ΠΕΡΙΣΤΕΡΙ', '2' : 'ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ' }
```

μπορούμε να έχουμε

```
keys_of_my_first_lexiko =  my_first_lexiko.keys()
print keys_of_my_first_lexiko
        dict_keys  ( ['ΑΡΧΗ'], ['ΕΡΟΜΕΝΟ'], ['2'] )
```

Μ.Ι.Σ. 2025-2026

```
values_of_my_first_lexiko =  my_first_lexiko.values()

print values_of_my_first_lexiko

        dict_values ( [ΑΘΗΝΑ, ΠΕΡΙΣΤΕΡΙ, ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ )


items_of_my_first_lexiko =  my_first_lexiko.items()

print items_of_my_first_lexiko

        dict_items([  (‘ΑΡΧΗ’, ΑΘΗΝΑ), (‘ΕΡΟΜΕΝΟ’, ΠΕΡΙΣΤΕΡΙ), (‘2’, ΑΓΙΑ ΠΑΡΑΣΚΕΥΗ)’  ])
```

Μ.Ι.Σ. 2025-2026

Αντίστοιχα, μπορούμε να διαπεράσουμε τον πίνακα-**λεξικό**, με την χρήση της εντολής **for**

```
keys_of_my_first_lexiko =  my_first_lexiko.keys()

values_of_my_first_lexiko =  my_first_lexiko.values()

items_of_my_first_lexiko =  my_first_lexiko.items()


for mykey in keys_of_my_first_lexiko :
        print (mykey)

for myvalue in values_of_my_first_lexiko :
        print (myvalue)

for mykey, myvalue in items_of_my_first_lexiko :
        print (mykey, myvalue)
```

καθώς υπάρχουν πολλές σχετικές συναρτήσεις για τα λεξικά-πίνακες και τον χειρισμό τους η εντολή **help(dict)** θα τυπώσει όλες τις διαθέσιμες επιλογές.

Μ.Ι.Σ. 2025-2026

Η ΕΝΝΟΙΑ ΤΗΣ ΑΝΑΦΟΡΑΣ ΣΕ ΜΙΑ ΜΕΤΑΒΛΗΤΗ

Στην γλώσσα PYTHON έχουμε την δυνατότητα να δηλώσουμε σε μια μεταβλητή και μια άλλη δεύτερη ονομασία, δημιουργώντας δηλαδή ένα δεύτερο όνομα για το ίδιο περιεχόμενο.

Μπορούμε δηλαδή να εκχωρήσουμε μια μεταβλητή σε μία άλλη, και έτσι και οι δύο θα αναφέρονται στο ίδιο περιεχόμενο.

Αυτό ονομάζεται **αναφορά**.

Έχοντας σε μια μεταβλητή δύο ονόματα, έχουμε την έννοια του **ψευδώνυμου**. Αλλαγές που γίνονται στο ένα ψευδώνυμο επηρεάζουν την ίδια μεταβλητή και με το άλλο όνομα.

Εάν κάνουμε χρήση του τελεστή **is** για δυο μεταβλητές που έχουν δυο διαφορετικά ονόματα (το ένα σαν αναφορά του άλλου) θα λάβουμε απάντηση ότι είναι δύο μεταβλητές που απλά αναφέρονται στο ίδιο περιεχόμενο.

```
my_a = [a, b, c, d]
my_b = my_a

print(my_a)
        [a, b, c, d]

print(my_b)
        [a, b, c, d]
my_a is my_b
        True
```

Μ.Ι.Σ. 2025-2026

Για να γίνει αντιληπτό, πως λειτουργεί μια αναφορά, μπορούμε να παρατηρήσουμε το πώς περνάμε μια λίστα σαν όρισμα σε μια συνάρτηση. Στην πραγματικότητα στις συναρτήσεις/μεθόδους/ρουτίνες περνάμε το περιεχόμενο της με μια αναφορά.

```
def  my_delete_routina(my_lista):
        del my_lista[0]

my_lista_nums = [1,2,3,4,5]

my_delete_routina(my_lista_nums)

print(my_lista_nums)
        [2,3,4,5]
```

Μ.Ι.Σ. 2025-2026